UNIVERSIDAD NACIONAL DE COLOMBIA

Estructuras de Datos – Semestre 2026-2

Residencias UNAL - Entrega 1

Integrantes

Nombre completo	**Correo institucional**
Daniel Felipe Morales Rojas	`danmoralesro@unal.edu.co`
John Alejandro Diaz Duran	`jodiazdu@unal.edu.co`
Daniel Poveda Ruiz	`dapovedar@unal.edu.co`

Descripción

La universidad tiene pocos cupos de residencia y muchos estudiantes que los solicitan. Este programa registra estudiantes (ID, nombre y puntaje socioeconómico) y asigna los cupos a quienes tienen **menor puntaje**, porque a menor puntaje hay mayor necesidad.

Funciones del programa:

1. Registrar estudiantes (el ID no se puede repetir).
2. Definir cuántos cupos hay.
3. Buscar un estudiante por su ID.
4. Listar los estudiantes de menor a mayor puntaje.
5. Asignar cupos y mostrar quiénes obtuvieron residencia y quiénes quedan en espera.
6. Modificar el puntaje de un estudiante (su posición se actualiza).
7. Eliminar un estudiante.

Estructuras de Datos

¿Por qué un árbol binario de búsqueda? Como se pueden modificar y eliminar estudiantes en cualquier momento, necesitamos una estructura que se mantenga ordenada

Estructura	Para qué se usa	Relación con el curso
Árbol binario de búsqueda (propio), clave = (`puntaje`, `ID`)	Mantener a los estudiantes ordenados por puntaje.	Terminología de árboles y recorrido inorden.
`HashMap` (tabla hash de Java, clave = `ID`)	Buscar por ID en $\mathcal{O}(1)$ promedio y evitar IDs repetidos.	Se usa como biblioteca; aún no se ha visto en clase.
`ArrayList` (arreglo dinámico)	Devolver las listas de estudiantes.	Arreglos dinámicos.

sin reordenar todo. Una lista ordenada cuesta $\mathcal{O}(n)$ por cada inserción o eliminación. En el árbol, el recorrido inorden (izquierda, nodo, derecha) entrega los estudiantes de menor a mayor puntaje.

¿Por qué la clave es (`puntaje`, `ID`)? Dos estudiantes pueden tener el mismo puntaje. Con el ID como desempate cada estudiante tiene una posición única.

Limitación conocida. El árbol no se balancea: si los puntajes llegan ya ordenados, se convierte en una lista y las operaciones cuestan $\mathcal{O}(n)$. El paso 9 de la demostración lo muestra (20 estudiantes ordenados dan altura 20 en vez de unos 5). Mejora prevista para una entrega posterior: usar un árbol balanceado (AVL).

Costo de cada operación (n = estudiantes, h = altura del árbol)

Operación	Costo	Promedio / Peor caso
Registrar	$\mathcal{O}(h)$	$\mathcal{O}(\log n)$ / $\mathcal{O}(n)$
Buscar por ID	$\mathcal{O}(1)$ promedio	Tabla hash
Modificar puntaje	$\mathcal{O}(h)$	$\mathcal{O}(\log n)$ / $\mathcal{O}(n)$ (se saca del árbol, se cambia y se vuelve a insertar)
Eliminar	$\mathcal{O}(h)$	$\mathcal{O}(\log n)$ / $\mathcal{O}(n)$
Listar ordenado	$\mathcal{O}(n)$	Recorrido inorden
Asignar cupos	$\mathcal{O}(n)$	Se recorre el árbol y se parte la lista en dos

Organización del Código

Todo está en `Main.java`, dividido en clases:

Clase	Responsabilidad
`Estudiante`	Datos de cada estudiante y cómo se comparan (puntaje, luego ID).
`Nodo`	Casilla del árbol.
`ArbolBinarioBusqueda`	Árbol ordenado por puntaje.
`SistemaResidencias`	Lógica: usa el `HashMap` y el árbol juntos.
`Main`	Menú de consola y demostración.

Cómo Ejecutarlo

Profe, es necesario tener JDK versión 17 o superior.